Sprint 3
Implementación real

Autenticación, permisos, estudiantes y matrículas conectados a PostgreSQL. Informe de implementación, pruebas y operación para Joel López, Product Owner.

Historia existente	Resultado técnico
SCRUM-23 / PB-011	Contraseñas con hash, sesión persistente, expiración, cierre y revocación.
SCRUM-24 / PB-012	Permisos de administración, docente y estudiante verificados en servidor.
SCRUM-27 / PB-015	Alta, consulta y edición persistentes; validaciones y auditoría.
SCRUM-28 / PB-016	Matrícula por curso/periodo, duplicidad, cupos y conservación de historial.

Tratamiento de Jira: 12 subtareas técnicas terminadas; 4 historias listas para revisión del PO. No se inventa una aceptación ni se cierra el sprint automáticamente. Se reutilizan sus incidencias e identificadores.

Alcance verificado: desarrollo local aislado con Node.js 22, PostgreSQL 14.22 y Chromium. La configuración Docker utiliza PostgreSQL 16, pero no se pudo ejecutar su motor en este Mac. El servidor remoto no fue modificado.

Unitarias: 4 pruebas aprobadas.

Integración: 23 pruebas aprobadas.

Navegador: 10 recorridos aprobados.

Build web: Verificado.

Dependencias API: Verificado.

Dependencias web: Verificado.

Compose local: Verificado.

Compose pruebas: Verificado.

Compose servidor: Verificado.

Fuentes de evidencia: código del repositorio; test-results.json; pruebas unitarias, integración y navegador incluidas en scripts y apps/*/test. Datos de pruebas totalmente sintéticos.

Arquitectura y permisos

Navegador - mismo origen web/Nginx - API Express - PostgreSQL. La API obtiene identidad y rol desde una sesión persistente; no acepta x-demo-role ni x-demo-student-id como autorización.

Operación	Administración	Docente	Estudiante
Estudiantes	Crear, editar y consultar	Consultar matriculados en cursos asignados	Consultar solo su registro
Matrículas	Crear, cerrar y consultar historial	Consultar cursos asignados	Consultar su historial
Cursos	Consultar estructura existente	Consultar asignaciones	Consultar su matrícula activa
Clases/asistencia (API)	No reemplaza al profesor	Crear/registrar solo con asignación vigente	Sin escritura
Notas publicadas	Fuera del gestor Sprint 3	Edición pendiente de integración	Solo notas propias publicadas
Auditoría	Consultar eventos	Sin acceso al registro general	Sin acceso al registro general

Las contraseñas **no se cifran de forma reversible**: se guardan con scrypt (N=131072, r=8, p=1), sal aleatoria y comparación de tiempo constante. No hay contraseñas comunes ni cuentas automáticas por defecto.

Sesiones opacas de 256 bits: solo su hash queda en la BD. Cookie HttpOnly y SameSite=Strict; en producción también Secure y prefijo __Host-. Duración máxima 8 horas e inactividad máxima 30 minutos. Cierre individual/global y cambio de contraseña revocan sesiones.

Protección de escrituras: origen explícito, JSON y token CSRF de sesión. Límite persistente de intentos de acceso y de concurrencia del hash. Errores de credenciales genéricos. Las respuestas sensibles no se almacenan en caché.

Referencias técnicas: Node.js 22 Crypto; OWASP Password Storage; OWASP Session Management.

Datos e historial

La migración 007 es aditiva: agrega sesiones, límites de intentos y referencias al usuario que crea/modifica estudiantes y matrículas. Conserva el esquema escolar existente; no vuelve a ejecutar la conversión antigua 003.

Recurso	Reglas aplicadas
Estudiante	Identificador normalizado, nombres/apellidos obligatorios, correo válido y único; estados controlados. Edición rechazada si la versión consultada ya cambió.
Matrícula	Estudiante regular, curso activo y periodo correcto/activo. Una matrícula activa por estudiante y periodo. Bloqueos transaccionales para evitar carreras y superar cupos.
Cierre	Retirada, finalizada o anulada; exige motivo. No se elimina el registro ni se reactiva un histórico: una nueva matrícula crea otro registro.
Auditoría	Usuario, acción, fecha, valores anteriores/nuevos. Se escribe en la misma transacción del cambio. La tabla existente impide modificar o eliminar sus eventos.
Persistencia	Prueba con procesos de API distintos: crear/editar, cerrar API, abrir nueva API y conexión, consultar mismos datos e historial.

Archivos principales: apps/api/src/security.js; school-app.js; repository.js; teacher-routes.js; manage-user.js; infra/postgres/init/007_sprint3_security.sql; apps/web/src/SecureApp.jsx.

Interfaz real: acceso común, panel por rol, estudiantes, cursos, matrículas, notas publicadas y seguridad de la cuenta. La edición de notas, alertas y justificaciones del prototipo no se mezcla con registros reales. App.jsx y prototype-data.js quedan como referencia histórica, no como entrada operativa.

La escritura docente de clases y asistencia tiene pruebas de autorización y persistencia en la API. Su interfaz completa, el motor configurable de alertas y el resto del Sprint 4 requieren su propia integración/revisión; este informe no los declara terminados.

Instalación local y cuentas

Requisitos: Git, Docker Desktop/Engine y Compose. Para desarrollo sin contenedores: Node.js 22.12 o superior y PostgreSQL. Trabajar desde la rama publicada del proyecto; no copiar claves del servidor a equipos de compañeros.

```
git clone https://github.com/joellopez775/SIGAA-Capstone-2026.git
cd "SIGAA-Capstone-2026/Fase 2/Evidencias Proyecto/SIGAA"
cp .env.example .env
# Configura POSTGRES_PASSWORD en .env con una clave propia.
docker compose up --build -d
```

En Windows copia .env.example como .env desde el explorador o con Copy-Item. Abre http://localhost:8088. Si cambias el puerto web, cambia también APP_ORIGIN: debe ser el origen exacto que usa el navegador.

Opción A: cuenta administrativa real. Guarda un JSON privado fuera de Git (.secrets/admin.json), con email, name, password y role=ADMINISTRADOR. Usa contraseña individual de al menos 12 caracteres. No incluyas ese archivo en commits.

```
docker compose exec -T api npm run user:create < .secrets/admin.json
```

El comando lee el secreto por entrada estándar y solo devuelve identificador y correo. Desde el portal la administración crea estudiantes y matrículas. Para crear una cuenta docente usa PROFESOR_ASIGNATURA; para ESTUDIANTE agrega studentId del estudiante existente. El vínculo es único. Las asignaciones de docentes y la estructura escolar deben estar previamente configuradas en la BD; su gestor general corresponde a otra historia.

Opción B: pruebas sintéticas, solo BD local vacía. No ejecutar después de la opción A. Genera cuentas y claves aleatorias nuevas; no existe una contraseña demo universal.

```
docker compose exec -e ALLOW_SAMPLE_DATA=yes \
  -e CREDENTIALS_FILE=/tmp/sigaa-accounts.json api npm run seed:local
docker compose cp api:/tmp/sigaa-accounts.json .secrets/credentials.json
```

Crea .secrets primero y limita el acceso a tu cuenta del sistema. Correos de prueba: admin@sigaa.local, docente@sigaa.local, estudiante@sigaa.local; sus contraseñas corresponden a admin, teacher y student en el archivo privado. El generador rechaza producción y bases con usuarios/estudiantes. En PowerShell utiliza Get-Content -Raw para pasar el JSON a user:create.

Migración del servidor

No ejecutada en el Mac servidor. Requiere revisión del responsable, respaldo recuperable, origen HTTPS y ventana de cambio. Este procedimiento no expone PostgreSQL ni concede SSH a compañeros.

1. Confirma el esquema escolar instalado y aplica las migraciones pendientes 004-006 si corresponde. No vuelvas a ejecutar 003 ni elimines volúmenes para actualizar. Respalda la base antes de continuar.

```
docker compose --env-file .env.server -f compose.server.yaml \
  exec -T db sh -c 'pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB"' \
  > respaldo-sigaa-antes-sprint3.sql
```

2. Mantén .env.server y secretos fuera de Git. Define APP_ORIGIN=https://tu-dominio-exacto. La API de producción rechaza un origen sin HTTPS y no permite usar cookies inseguras.

```
docker compose --env-file .env.server -f compose.server.yaml build api web
docker compose --env-file .env.server -f compose.server.yaml \
  run --rm --no-deps api npm run migrate
docker compose --env-file .env.server -f compose.server.yaml \
  up -d api web
```

3. Crea o vincula cuentas personales con user:create; verifica login, cierre de sesión, acceso entre dos usuarios, alta y matrícula, y consulta tras reiniciar la API. Conserva el respaldo cifrado fuera de Git. No publiques 5432 ni entregues DATABASE_URL a compañeros.

4. Si falla la verificación, no habilites usuarios reales. Conserva el respaldo y diagnostica. No vuelvas a publicar el login demo como solución. La migración es aditiva; una restauración debe hacerse en una base nueva y verificarse antes de sustituir la activa.

Límites de esta entrega: HTTPS entre navegador y servicio depende del túnel configurado. La red interna de Docker aísla la BD, pero no equivale a TLS entre API y PostgreSQL. No se declara ese cifrado interno implementado. El motor Docker no estuvo disponible para confirmar arranque de esta versión en contenedores.

Verificación y trazabilidad

Jira	Evidencia técnica
SCRUM-53 / 54 / 55	Hash con sal, error genérico, rotación, expiración, logout global y cambio de contraseña.
SCRUM-56 / 57 / 58	Matriz anterior; roles del servidor; pruebas 200/401/403, otro profesor/estudiante, cabeceras falsas y CSRF.
SCRUM-59 / 60 / 61	Lectura/escritura PostgreSQL, duplicidad y validaciones; reinicio real del proceso API y reconexión.
SCRUM-62 / 63 / 64	Reglas transaccionales, formulario conectado, cupos, duplicidad concurrente, cierre e historial.

```
cd SIGAA/apps/api
npm ci
npm test
TEST_DATABASE_URL=postgresql://usuario@127.0.0.1:5432/postgres \
  npm run test:integration
```

Usa un clúster de pruebas aislado: el ejecutor crea una base temporal única y la elimina al terminar. Nunca apuntes la prueba a la cuenta o instancia productiva. El usuario de prueba necesita CREATE DATABASE.

```
cd SIGAA
docker compose -f compose.test.yaml up --build \
  --abort-on-container-exit --exit-code-from api-tests
docker compose -f compose.test.yaml down
```

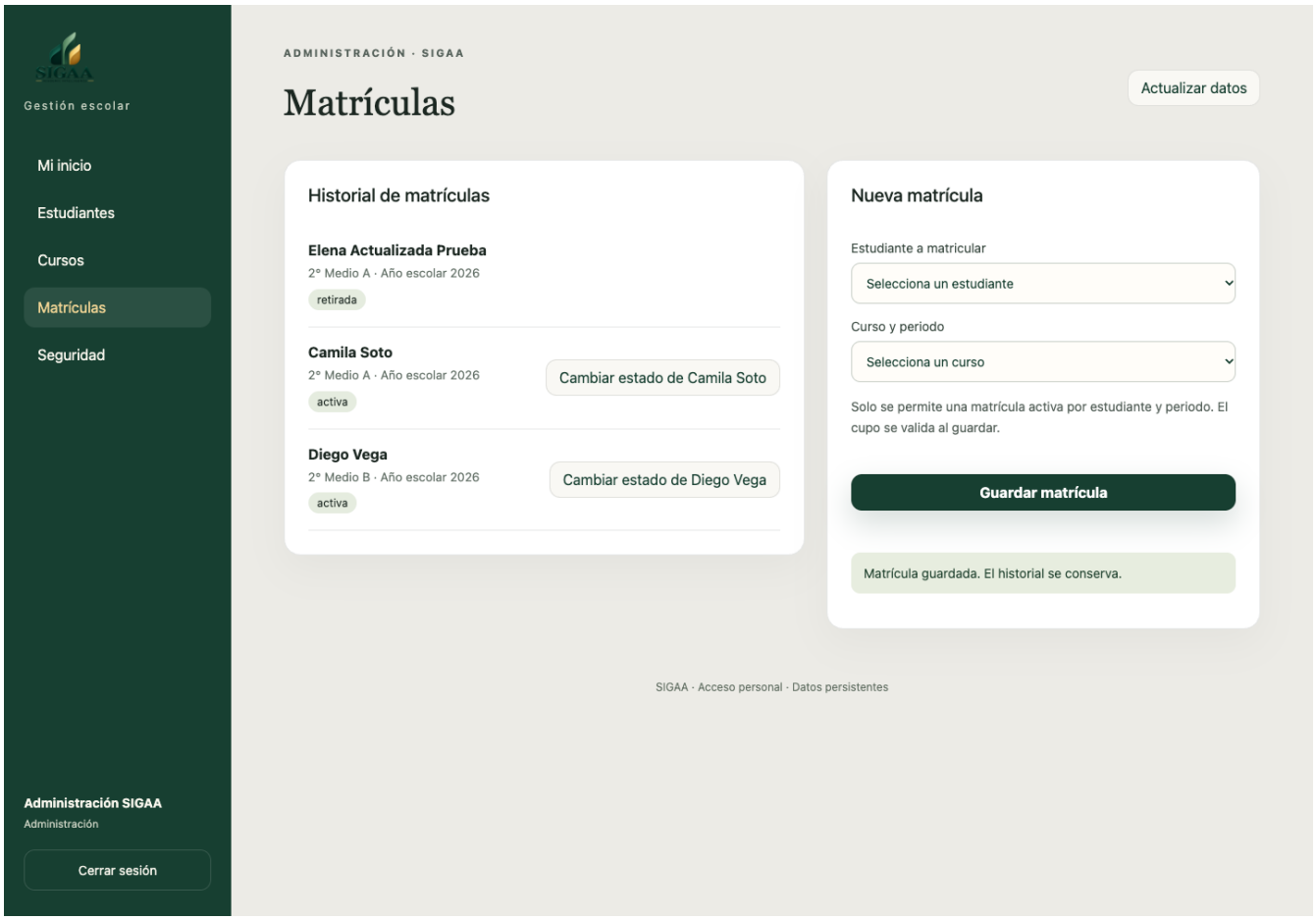
La alternativa Docker define un entorno efímero PostgreSQL 16 independiente y sin puertos publicados. Está preparada pero no ejecutada aquí. Para navegador, instala dependencias web y Chromium (npx playwright install chromium), y ejecuta scripts/verify-sprint3-browser.mjs con TEST_DATABASE_URL. CHROME_PATH permite elegir un Chrome instalado.

Cobertura observada: formularios, errores, permisos, recarga, logout, vista 1440×1000 y móvil 390×844. 10 recorridos de navegador y cero errores JavaScript. No se afirma compatibilidad con Safari/Firefox ni conformidad WCAG completa.

Recomendación QA: requiere verificaciones antes de producción. El incremento local pasó su cobertura funcional; faltan validar contenedores PostgreSQL 16, respaldo/restauración y despliegue HTTPS del servidor. Listados limitados a 500 registros: añadir paginación antes de exceder ese volumen. No se efectuaron pruebas de carga ni un pentest integral.

Evidencia visual

Formulario de matrícula real: el cierre conserva el registro y su estado. Datos sintéticos de la ejecución automatizada.



Vista móvil

Vista de estudiante y navegación desplegable a 390 px. Identidad y alcance vienen de la sesión del servidor.

